



Curso de Assimilação de Dados para Previsão Numérica de Tempo e Clima



Curso de Assimilação de Dados Atmosféricos

Carga horária sugerida: 40–60 horas

Formato: Aulas teóricas + exercícios práticos
(Python/Fortran) + estudo de casos reais



Módulo 3 — Métodos clássicos

1. Interpolação Ótima (OI)

1. Equação de análise de Kalman estático
2. Exemplo numérico simples

2. 3D-Var

1. Minimização da função custo
2. Métodos numéricos para otimização
3. Implementação prática com um modelo simples

3. Filtro de Kalman (KF)

1. Propagação e atualização
2. Limitações



Módulo 3 — Métodos Clássicos de Assimilação de Dados



Módulo 3 — Métodos Clássicos de Assimilação de Dados

1.2 Características da OI

- **Método relativamente simples.**
- **Assume que os erros de modelo e observação são gaussianos e independentes.**
- **Só é adequado quando as observações estão distribuídas de forma esparsa e a relação entre o modelo e as observações é linear.**



“Notação generalizada”

Estado do modelo (análise):

“Primeira estimativa” (First Guess):

Observações:

“Operador direto” H:

vetor do modelo x ($n \approx 10^7$ a 10^9 graus de liberdade)

previsão de curto prazo x^{fg}

vetor y ($m \approx 10^4$ a 10^7 observações / janela temporal)

permite simular as observações: $y^{sim} = H(x^{fg})$

“Minimização da função de penalidade $J(x)$ ”

$$J(x) = \underbrace{\frac{1}{2} \sum_{p=1}^n \frac{(x_p - x_p^{fg})^2}{(\sigma_p^{fg})^2}}_{\text{Minimiza a distância entre (fg) e a análise}} + \underbrace{\frac{1}{2} \sum_{q=1}^m \frac{(H_q(x) - y_q)^2}{(\sigma_q^{obs})^2}}_{\text{Minimiza a distância entre (OBS) e a análise}}$$

onde:

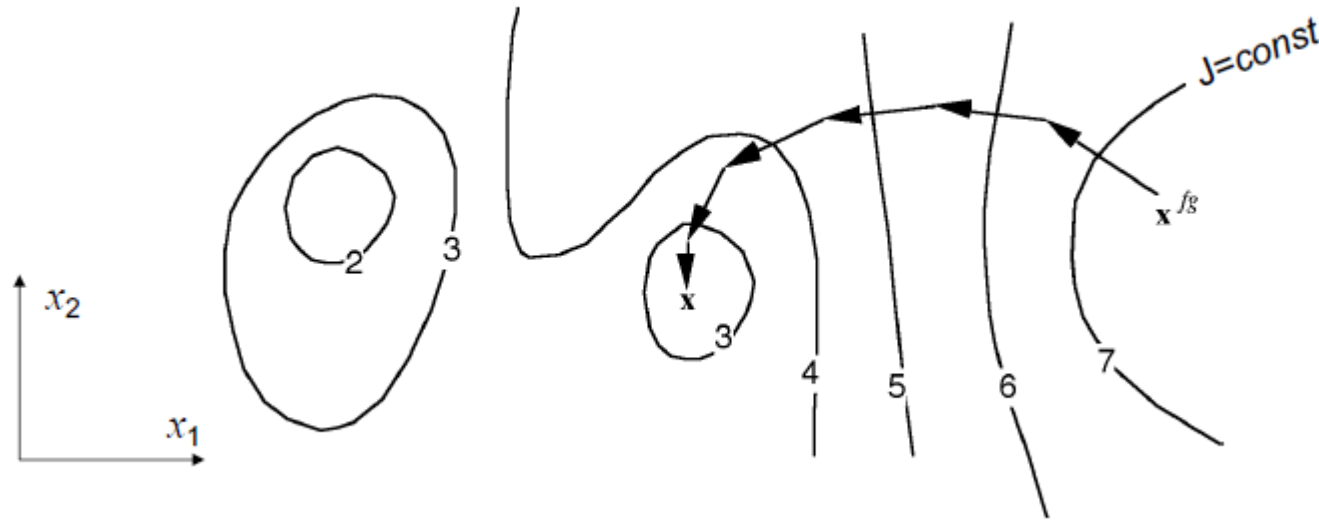
- $(\sigma_q^{obs})^2$ = variância do erro das observações, incluindo
 - (i) erros observacionais e
 - (ii) erros do operador direto (incluindo interpolação)
- $(\sigma_p^{fg})^2$ = variância do erro da primeira estimativa (first guess)

As estimativas estatísticas são derivadas do desempenho passado das previsões



“Minimização na assimilação variacional”

“Minimização da função de penalidade $J(x)$ iniciando na primeira estimativa x_p^{fg} ”



“A minimização de $J(x)$ requer o cálculo do gradiente”

$$J(x) = \frac{1}{2} \sum_{p=1}^n \frac{(x_p - x_p^{fg})^2}{(\sigma_p^{fg})^2} + \frac{1}{2} \sum_{q=1}^m \frac{(H_q(x) - y_q)^2}{(\sigma_q^{obs})^2}$$

$$[\nabla J(\mathbf{x})]_p = \frac{x_p - x_p^{fg}}{(\sigma_p^{fg})^2} + \sum_{q=1}^m \frac{H_q(\mathbf{x}) - y_q}{(\sigma_q^{obs})^2} \frac{\partial H_q(\mathbf{x})}{\partial x_p}$$



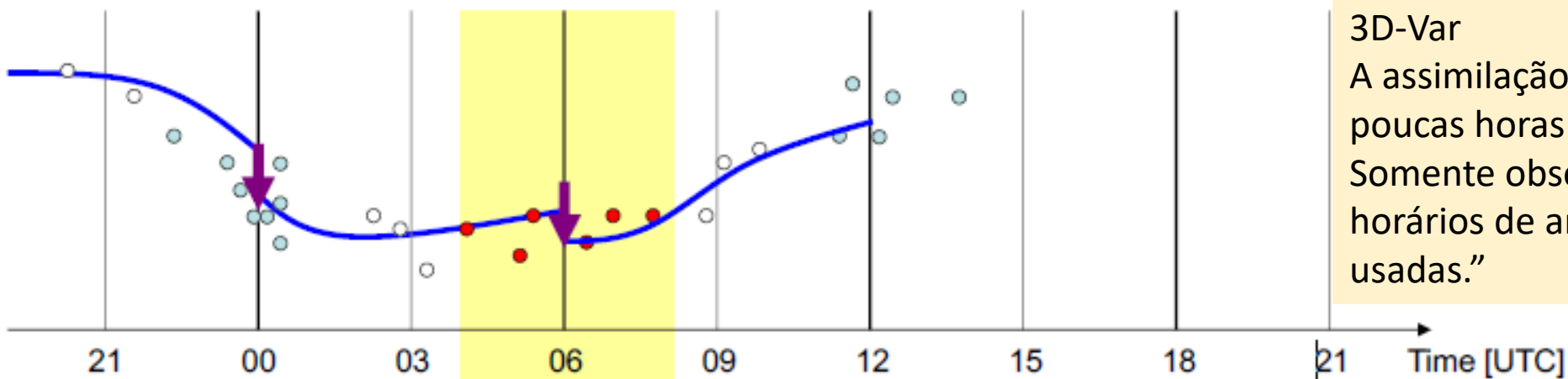
$$J(x) = \frac{1}{2} \sum_{p=1}^n \frac{(x_p - x_p^{fg})^2}{(\sigma_p^{fg})^2} + \frac{1}{2} \sum_{q=1}^m \frac{(H_q(x) - y_q)^2}{(\sigma_q^{obs})^2}$$

“A minimização de $J(x)$ requer o cálculo do gradiente”

$$[\nabla J(\mathbf{x})]_p = \frac{x_p - x_p^{fg}}{(\sigma_p^{fg})^2} + \sum_{q=1}^m \frac{H_q(\mathbf{x}) - y_q}{(\sigma_q^{obs})^2} \frac{\partial H_q(\mathbf{x})}{\partial x_p}$$

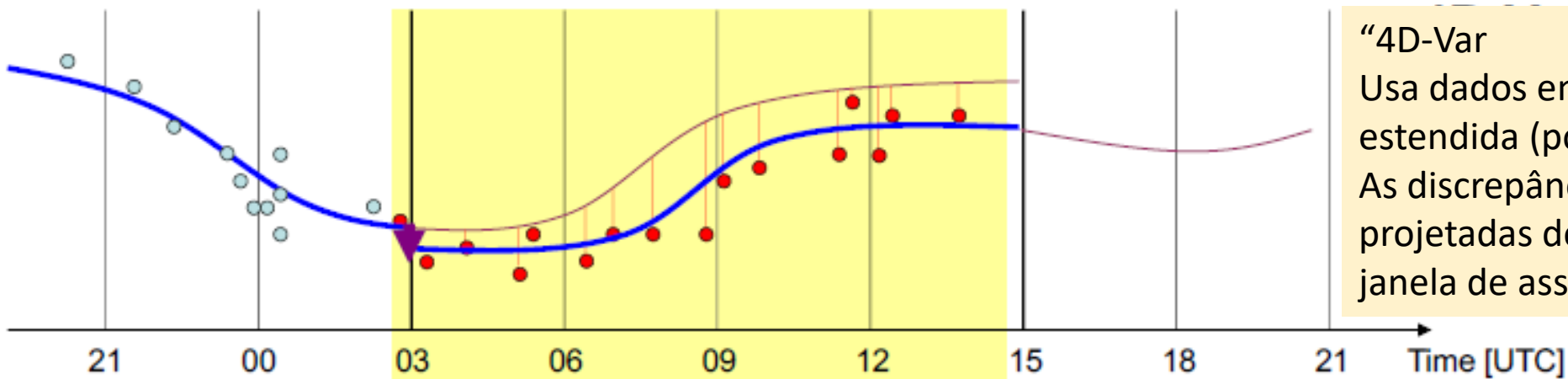


“Assimilação 3D-Var versus 4D-Var”



3D-Var

A assimilação é realizada a cada poucas horas (por exemplo, 6 h). Somente observações próximas aos horários de análise podem ser usadas.”



“4D-Var

Usa dados em uma janela de tempo estendida (por exemplo, 12 h). As discrepâncias (OBS-FG) são projetadas de volta para o início da janela de assimilação.”

— Model trajectory ↓ Assimilation increment ● Observations



“Assimilação 3D-Var versus 4D-Var”



$$J(x) = \frac{1}{2} \sum_{p=1}^n \frac{(x_p - x_p^{fg})^2}{(\sigma_p^{fg})^2} + \frac{1}{2} \sum_{q=1}^m \frac{(H_q(x) - y_q)^2}{(\sigma_q^{obs})^2}$$

“A soma p percorre todos os graus de liberdade do modelo.
A soma q percorre todas as observações disponíveis.”

3D-Var

A assimilação é realizada a cada poucas horas (por exemplo, 6 h).
Somente observações próximas aos horários de análise podem ser usadas.”

$$J(\mathbf{x}^o) = \frac{1}{2} \sum_{i=0}^T \sum_{p=1}^n \frac{(x_p^i - x_p^{fg,i})^2}{(\sigma_p^{fg})^2} + \frac{1}{2} \sum_{i=0}^T \sum_{q=1}^{m_i} \frac{(H_{i,q}(\mathbf{x}^i) - y_q^i)^2}{(\sigma_q^{obs,i})^2}$$

“A soma i percorre todas as janelas de tempo (com duração, por exemplo, de 1 h),
onde x^0 e x^i define o estado do modelo nos instantes $t = t_0$ e $t = t_i$, respectivamente.
A minimização no 4D-Var envolve a aproximação linear tangente.”

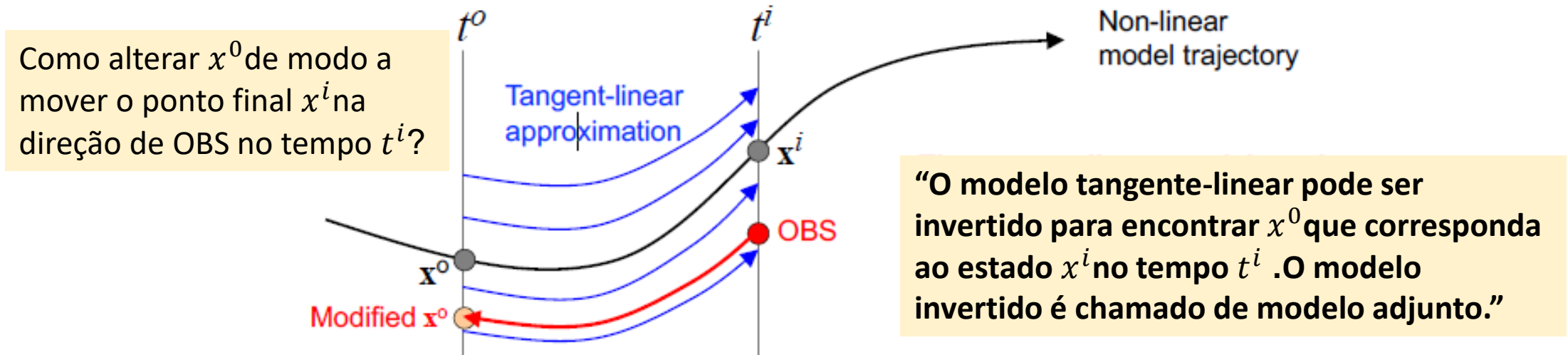
“4D-Var

Usa dados em uma janela de tempo estendida (por exemplo, 12 h).
As discrepâncias (OBS-FG) são projetadas de volta para o início da janela de assimilação.”



“Aproximação linear tangente no 4D-Var”

“Para conectar diferentes tempos, faz-se a aproximação linear tangente, que se refere à linearização em torno de uma trajetória particular do modelo não linear.”



A minimização requer o cálculo do **gradiente** $\nabla J(x_0)$, e isso incluirá termos da forma:

$$(A^i)_{p,l} = \frac{\partial x_p^i}{\partial x_l^0}$$

(os termos resultam da aplicação da regra da cadeia)

O termo descreve **como o estado do modelo x^i no tempo t^i depende de x^0 no tempo t^0** .



Ensemble Kalman Filter (EnKF)

- **Ensemble Kalman Filter (EnKF)**: used e.g. in Canadian global model and the MeteoSwiss regional model. It uses an ensemble of model simulations to obtain better estimates of the uncertainties. The mathematical details are intricate.
 - The method is based on the conventional **Kalman Filter (KF)**, which has numerous technological applications. Common applications include tools for guidance, navigation, and control of vehicles – particularly aircraft, spacecraft and ships (see https://en.wikipedia.org/wiki/Kalman_filter). The Kalman Filter has two steps:
 1. Forward integration of system using dynamical equations
 2. Correction of system state, using observations and observational uncertainties
- The KF is thus logically related to 3D-Var, but it employs a better estimate of the error covariance σ_p^{fg} . In particular, σ_p^{fg} is not constant in time, but depends on the state of the system.
- The **EnKF** uses an ensemble of forward integrations perturbed by observational uncertainties to make the procedure more tractable for highly complicated systems (such as NWP models).



Nudging

Nudging is an early (and partly outdated) assimilation technique. It is still used for some applications today (e.g. assimilation of SST data into ocean models).

The prognostic variables are “pulled” towards the observations by nudging terms during the integration of the forecasting model:

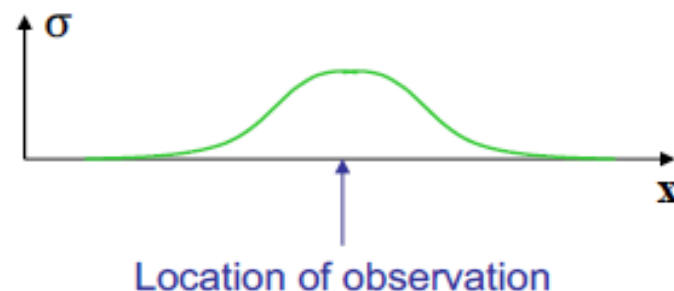
$$\underbrace{\frac{\partial \chi}{\partial t}}_{\text{“governing equations”}} = \text{physical terms} + \underbrace{\sum_i \varepsilon_i \sigma_i (\mathbf{x} - \mathbf{x}_i, t - t_i) (\chi_i^{obs} - \chi(\mathbf{x}_i, t_i))}_{\text{Nudging terms}}$$

where

ε_i = weight of observation i

σ_i = weight in space and time

(the choice of these weights is a bit heuristic)





Módulo 3 — Métodos Clássicos de Assimilação de Dados

2. Método 3D-Var

O **3D-Var** é um método de otimização que busca minimizar a função custo que descreve a discrepância entre as previsões do modelo e as observações. Esse método considera uma janela de tempo fixo (daí o "3D" - três dimensões: latitude, longitude e tempo).

2.1 Formulação da Função Custo

A função custo para o **3D-Var** é dada por:

$$J(x) = (x - x_b)^T B^{-1} (x - x_b) + (y - Hx)^T R^{-1} (y - Hx)$$

O objetivo é encontrar o $\mathbf{x}$ que minimize $J(x)$, ou seja, a combinação ótima de $\mathbf{x}_b$ (o estado do modelo) e $\mathbf{y}$ (as observações). Esse processo é realizado por um método iterativo, como o **gradiente descendente**.



Módulo 3 — Métodos Clássicos de Assimilação de Dados

2.2 Solução

A solução para a análise $x_{\hat{a}}$ no 3D-Var é obtida resolvendo a seguinte equação:

$$x_{\hat{a}} = x_b + BH^T (HBH^T + R)^{-1} (y - Hx_b)$$

Isso é muito parecido com a fórmula da OI, mas o **3D-Var** pode ser aplicado de forma mais flexível para dados em 3D (ou seja, considerando dependências temporais e espaciais).

2.3 Características do 3D-Var

- Melhor para sistemas dinâmicos complexos.
- Pode ser aplicado a dados temporais e 3D.
- O principal desafio é a otimização da função custo, que pode ser feita por métodos como o **gradiente descendente**.



Módulo 3 — Métodos Clássicos de Assimilação de Dados

3. Filtro de Kalman (KF)

O **Filtro de Kalman** é um algoritmo recursivo que utiliza informações passadas e atuais para estimar o estado de um sistema dinâmico. É particularmente útil quando há incertezas nos dados e no modelo.

3.1 Formulação do Filtro de Kalman

O Filtro de Kalman pode ser dividido em duas etapas principais: **previsão** e **atualização**.

1. Previsão:

A previsão do estado do sistema $\hat{x}_{k|k-1}$ no passo k é dada por:

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1|k-1} + Bu_k$$

1. onde **F é a matriz de transição do estado**, $\hat{x}_{k-1|k-1}$ **é o estado estimado no passo anterior**, e u_k é o controle aplicado no sistema.



Módulo 3 — Métodos Clássicos de Assimilação de Dados

2 Atualização:

A atualização do estado é feita com base nas observações y_k no passo k :

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(y_k - H\hat{x}_{k|k-1})$$

1. onde:

1. K_k é o **ganho de Kalman**, calculado como:

$$K_k = P_{k|k-1}H^T(H P_{k|k-1}H^T + R)^{-1}$$

1. onde $P_{k|k-1}$ é a matriz de covariância do erro da previsão.
2. O termo $(y_k - H\hat{x}_{k|k-1})$ é a **inovação**, ou a diferença entre a observação e a previsão do modelo.



Módulo 3 — Métodos Clássicos de Assimilação de Dados

3.2 Características do Filtro de Kalman

- O Filtro de Kalman é eficiente e ideal para sistemas lineares.
- Ele **atualiza a previsão do estado com base nas observações de forma recursiva**, utilizando o **ganho de Kalman**.
- A principal limitação do Filtro de Kalman tradicional é que ele assume que os erros do modelo e da observação são gaussianos e que a dinâmica do sistema é linear.



Módulo 3 — Métodos Clássicos de Assimilação de Dados

Exemplo Prático: Implementação do 3D-Var em Python

Vamos implementar o **3D-Var** em Python para uma assimilação simples de temperatura.

Exemplo de Dados:

Temos um modelo com valores estimados de temperatura e uma observação no passo k .



Módulo 3 — Métodos Clássicos de Assimilação de Dados

```
import numpy as np

# Dados do modelo (temperatura estimada)
x_b = np.array([10, 12, 14, 16, 18, 20]) # Temperatura no modelo

# Observação
y = np.array([15.5]) # Observação de temperatura

# Matriz de covariância do erro de modelo (B)
B = np.array([[1]])

# Matriz de covariância do erro de observação (R)
R = np.array([[0.5]])

# Operador de observação (H) - neste caso, vamos usar o valor H = [0, 0, 1, 0, 0, 0]
H = np.array([[0, 0, 1, 0, 0, 0]])

# Calculando a análise
Ht = H.T
B_inv = np.linalg.inv(B)
R_inv = np.linalg.inv(R)

# Fórmula da análise (ajuste do modelo)
x_hat = x_b + np.dot(np.dot(B_inv, Ht), np.linalg.inv(np.dot(np.dot(H, B_inv), Ht) + R_inv).dot(y - np.dot(H, x_b)))

print("Estado corrigido após assimilação:", x_hat)
```

Exercício 3:

1. Implemente o **Filtro de Kalman** para um sistema simples de previsão e atualização, usando um modelo linear.
2. Aplique o **3D-Var** a um modelo mais complexo, incluindo mais variáveis (temperatura, umidade, vento) e diferentes tipos de observações.
3. Teste o código com diferentes matrizes de covariância e analise os resultados.



Módulo 3 — Métodos Clássicos de Assimilação de Dados

Conclusão do Módulo 3

- Os métodos clássicos de assimilação de dados, como a **Interpolação Ótima (OI)**, **3D-Var**, e **Filtro de Kalman (KF)**, são fundamentais para ajustar os modelos atmosféricos e melhorar as previsões.
- O **3D-Var** é mais robusto para modelos dinâmicos, e o **Filtro de Kalman** é eficaz para sistemas lineares com incertezas.
- Nos próximos módulos, abordaremos **Métodos Avançados** e **Aplicações Práticas** desses métodos.



Módulo 3 — Métodos Clássicos de Assimilação de Dados

Vamos gerar os exemplos de código **Fortran** para os métodos clássicos que discutimos no Módulo 3: **Interpolação Ótima (OI)**, **3D-Var**, e **Filtro de Kalman (KF)**. Vamos estruturar os exemplos para que você possa usá-los diretamente nas aulas.

Exemplo 1: Interpolação Ótima (OI) em Fortran

Este código implementa a **Interpolação Ótima (OI)**, que ajusta o modelo baseado nas observações.

Código Fortran para OI



```
program interpolacao_otima
implicit none
integer, parameter :: n = 6
real(8) :: x_b(n), y, H(n), Ht(n), B_inv, R_inv
real(8) :: x_hat(n), B(n, n), R, H_B_inv_Ht, term
integer :: i
! Definindo os dados do modelo (temperatura estimada)
x_b = [10.0d0, 12.0d0, 14.0d0, 16.0d0, 18.0d0, 20.0d0]
! Observação
y = 15.5d0
! Matriz de covariância do erro de modelo (B) (assumindo uma matriz diagonal para simplificação)
B = 1.0d0 ! Matriz identidade para simplificação
B_inv = 1.0d0 ! Inversa de B (aqui simplificada)

! Matriz de covariância do erro de observação (R)
R = 0.5d0
R_inv = 1.0d0 / R ! Inversa de R
! Operador de observação (H) - neste caso, vamos usar o valor H = [0, 0, 1, 0, 0, 0]
H = [0.0d0, 0.0d0, 1.0d0, 0.0d0, 0.0d0, 0.0d0]

! Calculando Ht (transposta de H, mas como H é uma linha, é igual a H)
Ht = H
! Multiplicando H*B_inv*Ht
H_B_inv_Ht = 0.0d0
do i = 1, n
    H_B_inv_Ht = H_B_inv_Ht + H(i) * B_inv * Ht(i)
end do

! Calculando a análise (x_hat) utilizando a fórmula
term = y - matmul(H, x_b)
x_hat = x_b + B_inv * matmul(Ht, 1.0d0 / (H_B_inv_Ht + R_inv) * term)

! Exibindo o resultado da análise
print *, "Estado corrigido após assimilação: ", x_hat
end program interpolacao_otima
```

Explicação:

- Este código implementa a fórmula da **Interpolação Ótima (OI)** para um modelo simples de temperatura, ajustando o valor de um vetor **x_b** com base em uma observação **y** .
- A multiplicação da inversa da matriz **B** e o cálculo de $H \cdot B^{-1} \cdot H^T$ são feitos por um loop, e o valor de **x_{hat}** é calculado pela fórmula de análise.



Exemplo 2: 3D-Var em Fortran

Agora, vamos implementar o **3D-Var**, que também ajusta o estado do modelo, mas leva em consideração uma janela de tempo.

Código Fortran para 3D-Var



```
program metodo_3DVar
implicit none
integer, parameter :: n = 6
real(8) :: x_b(n), y, H(n), Ht(n), B_inv, R_inv
real(8) :: x_hat(n), B(n, n), R, H_B_inv_Ht, term
integer :: i
! Definindo os dados do modelo (temperatura estimada)
x_b = [10.0d0, 12.0d0, 14.0d0, 16.0d0, 18.0d0, 20.0d0]
! Observação
y = 15.5d0
! Matriz de covariância do erro de modelo (B)
B = 1.0d0 ! Matriz identidade
B_inv = 1.0d0 ! Inversa de B

! Matriz de covariância do erro de observação (R)
R = 0.5d0
R_inv = 1.0d0 / R ! Inversa de R

! Operador de observação (H)
H = [0.0d0, 0.0d0, 1.0d0, 0.0d0, 0.0d0, 0.0d0]

! Calculando Ht (transposta de H, aqui é igual a H)
Ht = H

! Multiplicação de H*B_inv*Ht
H_B_inv_Ht = 0.0d0
do i = 1, n
    H_B_inv_Ht = H_B_inv_Ht + H(i) * B_inv * Ht(i)
end do

! Calculando a análise (x_hat) utilizando a fórmula de 3D-Var
term = y - matmul(H, x_b)
x_hat = x_b + B_inv * matmul(Ht, 1.0d0 / (H_B_inv_Ht + R_inv) * term)

! Exibindo o resultado da análise
print *, "Estado corrigido após assimilação: ", x_hat
end program metodo_3DVar
```

Explicação:

- Este exemplo segue a mesma estrutura do exemplo de **OI**, mas se refere ao método **3D-Var**.
- A fórmula é praticamente idêntica à de OI, mas o conceito de janela de tempo é mais aplicável em cenários com dados temporais. O **3D-Var** funciona de forma similar, ajustando o estado **x_b** baseado em uma observação **y**.



Exemplo 3: Filtro de Kalman (KF) em Fortran

Finalmente, vamos implementar o **Filtro de Kalman**, que é um algoritmo recursivo e eficiente para sistemas dinâmicos.

Código Fortran para Filtro de Kalman (KF)



```
program filtro_kalman
implicit none
integer, parameter :: n = 6
real(8) :: x_b(n), x_hat(n), y, H(n), P(n, n), P_pred(n, n), K(n), R, B_inv
real(8) :: Ht(n), P_inv(n, n), term
integer :: i

! Definindo os dados do modelo (temperatura estimada)
x_b = [10.0d0, 12.0d0, 14.0d0, 16.0d0, 18.0d0, 20.0d0]

! Observação
y = 15.5d0

! Matriz de covariância do erro de modelo (P)
P = 1.0d0 ! Inicialização com a identidade

! Matriz de covariância do erro de observação (R)
R = 0.5d0

! Operador de observação (H)
H = [0.0d0, 0.0d0, 1.0d0, 0.0d0, 0.0d0, 0.0d0]

! Predição do estado
P_pred = P ! Para simplificação, assumimos que a predição de P é igual a P

! Cálculo do ganho de Kalman (K)
K = P_pred * H / (H * P_pred * H + R)

! Atualizando o estado (x_hat) com a inovação (y - H * x_b)
term = y - matmul(H, x_b)
x_hat = x_b + K * term

! Exibindo o resultado da análise
print *, "Estado corrigido após assimilação (Filtro de Kalman): ", x_hat
end program filtro_kalman
```

Explicação:

- O código acima implementa o **Filtro de Kalman** para o caso simples de um modelo de temperatura.
- A etapa de **previsão** e **atualização** é feita de forma recursiva. O ganho de Kalman **K** é calculado e utilizado para atualizar o estado do modelo com base na diferença entre a observação e a previsão.

$$K_k = P_{k|k-1} H^T (H P_{k|k-1} H^T + R)^{-1}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (y_k - H \hat{x}_{k|k-1})$$



Conclusão

Esses exemplos de código Fortran implementam os métodos clássicos de assimilação de dados atmosféricos:

- 1. Interpolação Ótima (OI) – Ajusta o modelo com base em uma única observação.**
- 2. 3D-Var – Similar ao OI, mas usado para dados temporais**